
Project: CNN Acceleration System

| 2025 – 2 | SoC | 10조 : 김민성 오현석 이원우 차재민

Overview

- System Architecture
- Method
 - PE Array
 - Line buffer
 - Reconfigurable PE
 - Requantization & Max Pooling
 - FC Layer & Comparator
 - Pipelining
- FPGA Implementation Results



Overview

- System Architecture
- Method
 - PE Array
 - Line buffer
 - Reconfigurable PE
 - Requantization & Max Pooling
 - FC Layer & Comparator
 - Pipelining
- FPGA Implementation Results

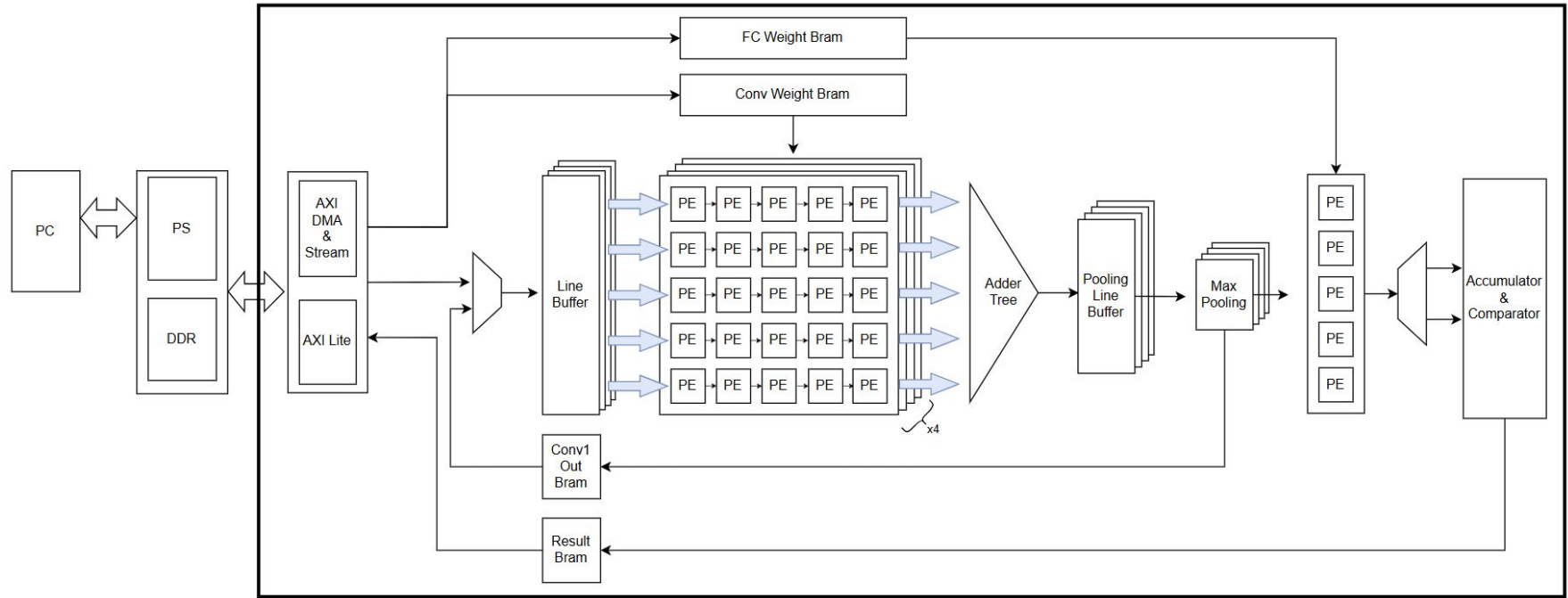


Overview

- System Architecture
- Method
 - PE Array
 - Line buffer
 - Reconfigurable PE
 - Requantization & Max Pooling
 - FC Layer & Comparator
 - Pipelining
- **FPGA Implementation Results**

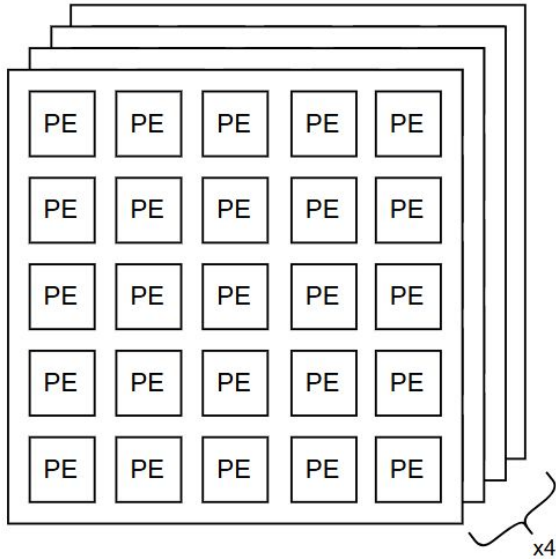


System Architecture

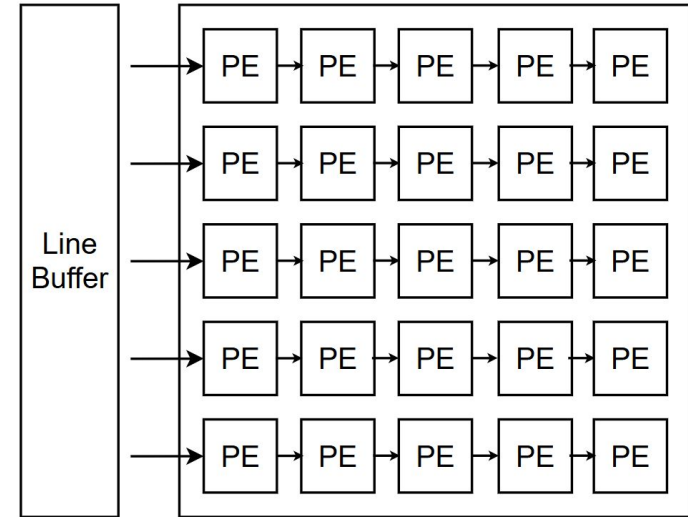


Method

PE Array



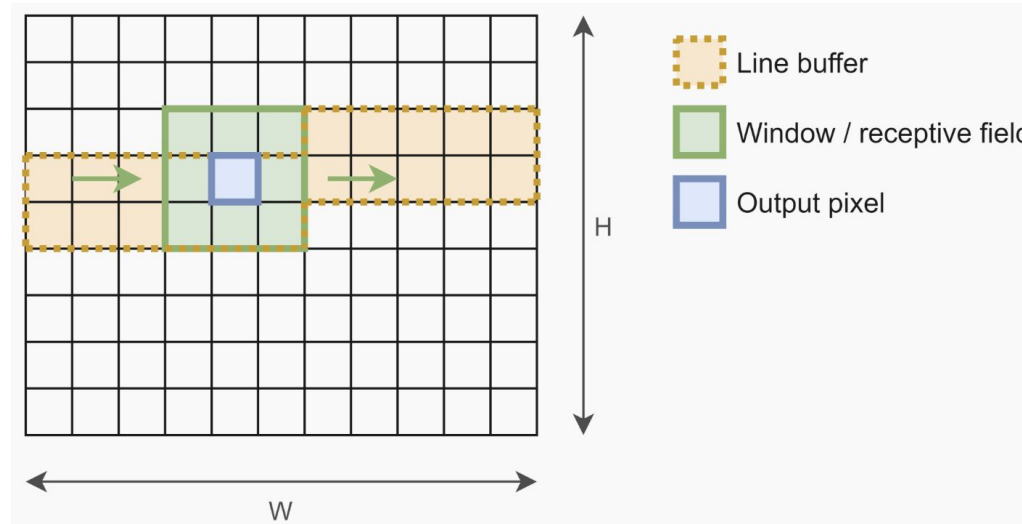
Multiplier 5x5x4



Weight stationary

Method

Line buffer



A streaming approach that receives an image pixel by pixel can make effective use of a line buffer.

Reference: [A tutorial on non-separable 2D convolutions in Vivado HLS – Basile Van Hoorick](#)

Method

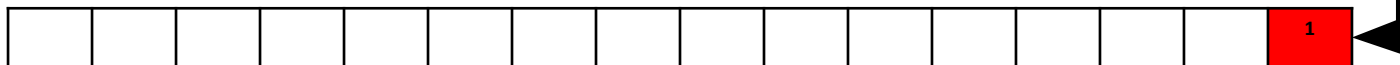
Line buffer - Conv1

Input Image Pixels

1	2	3	4	5	6	...	27	28
29	30	31	32	33	34	...	55	56
57	58	59	60	61	62	...	83	84
85	86	87	88	89	90	...	111	112
113	114	115	116	117	118	...	139	140
141	142	143	144	145	146	...	167	168
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
729	730	731	732	733	734	...	755	756
757	758	759	760	761	762	...	783	784

1. The line buffer is filled with 4 rows.

Line Buffer



$$28 * 4$$

Method

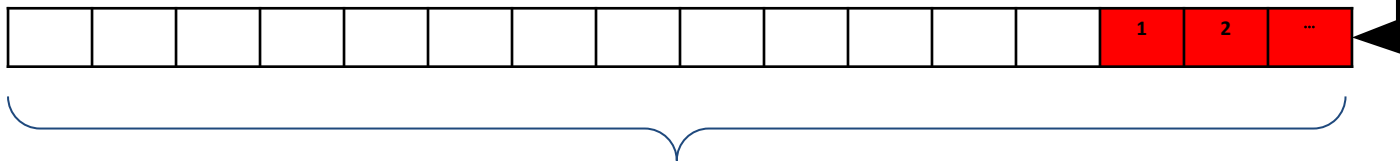
Line buffer - Conv1

Input Image Pixels

1	2	3	4	5	6	...	27	28
29	30	31	32	33	34	...	55	56
57	58	59	60	61	62	...	83	84
85	86	87	88	89	90	...	111	112
113	114	115	116	117	118	...	139	140
141	142	143	144	145	146	...	167	168
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
729	730	731	732	733	734	...	755	756
757	758	759	760	761	762	...	783	784

1. The line buffer is filled with 4 rows.

Line Buffer



$$28 * 4$$

Method

Line buffer - Conv1

Input Image Pixels

1	2	3	4	5	6	...	27	28
29	30	31	32	33	34	...	55	56
57	58	59	60	61	62	...	83	84
85	86	87	88	89	90	...	111	112
113	114	115	116	117	118	...	139	140
141	142	143	144	145	146	...	167	168
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
729	730	731	732	733	734	...	755	756
757	758	759	760	761	762	...	783	784

1. The line buffer is filled with 4 rows.

Line Buffer



$$28 * 4$$

Method

Line buffer - Conv1

Input Image Pixels

1	2	3	4	5	6	...	27	28
29	30	31	32	33	34	...	55	56
57	58	59	60	61	62	...	83	84
85	86	87	88	89	90	...	111	112
113	114	115	116	117	118	...	139	140
141	142	143	144	145	146	...	167	168
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
729	730	731	732	733	734	...	755	756
757	758	759	760	761	762	...	783	784

1. The line buffer is filled with 4 rows.

Line Buffer

1	2	...	28	29	30	...	56	57	58	...	84	85	86	...	112
---	---	-----	----	----	----	-----	----	----	----	-----	----	----	----	-----	-----

$$28 * 4$$

Method

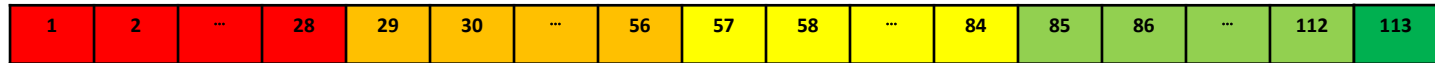
Line buffer - Conv1

Input Image Pixels

1	2	3	4	5	6	...	27	28
29	30	31	32	33	34	...	55	56
57	58	59	60	61	62	...	83	84
85	86	87	88	89	90	...	111	112
113	114	115	116	117	118	...	139	140
141	142	143	144	145	146	...	167	168
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
729	730	731	732	733	734	...	755	756
757	758	759	760	761	762	...	783	784

2. The next input pixel arrives.

Line Buffer



$$28 * 4$$

Method

Line buffer - Conv1

Input Image Pixels

1	2	3	4	5	6	...	27	28
29	30	31	32	33	34	...	55	56
57	58	59	60	61	62	...	83	84
85	86	87	88	89	90	...	111	112
113	114	115	116	117	118	...	139	140
141	142	143	144	145	146	...	167	168
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
729	730	731	732	733	734	...	755	756
757	758	759	760	761	762	...	783	784

Line Buffer

1	2	...	28	29	30	...	56	57	58	...	84	85	86	...	112	113
---	---	-----	----	----	----	-----	----	----	----	-----	----	----	----	-----	-----	-----

Convolution

				1
				29
				57
				85
				113

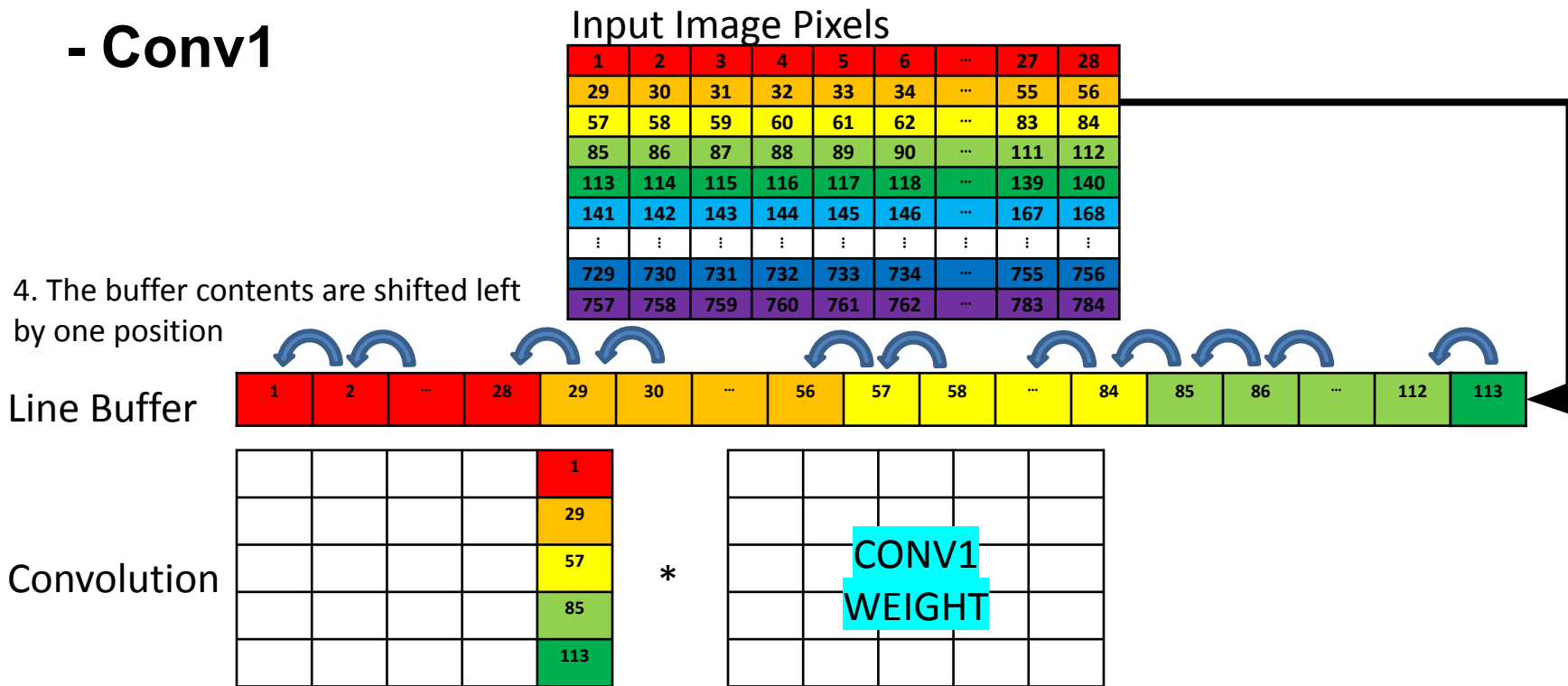
*

CONV1
WEIGHT

3. The pixel at the specified index appears at the output

Method

Line buffer - Conv1



Method

Line buffer - Conv1

Input Image Pixels

1	2	3	4	5	6	...	27	28
29	30	31	32	33	34	...	55	56
57	58	59	60	61	62	...	83	84
85	86	87	88	89	90	...	111	112
113	114	115	116	117	118	...	139	140
141	142	143	144	145	146	...	167	168
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
729	730	731	732	733	734	...	755	756
757	758	759	760	761	762	...	783	784

Line Buffer

2	3	...	29	30	...	56	57	58	...	84	85	86	...	112	113	114
---	---	-----	----	----	-----	----	----	----	-----	----	----	----	-----	-----	-----	-----

Convolution

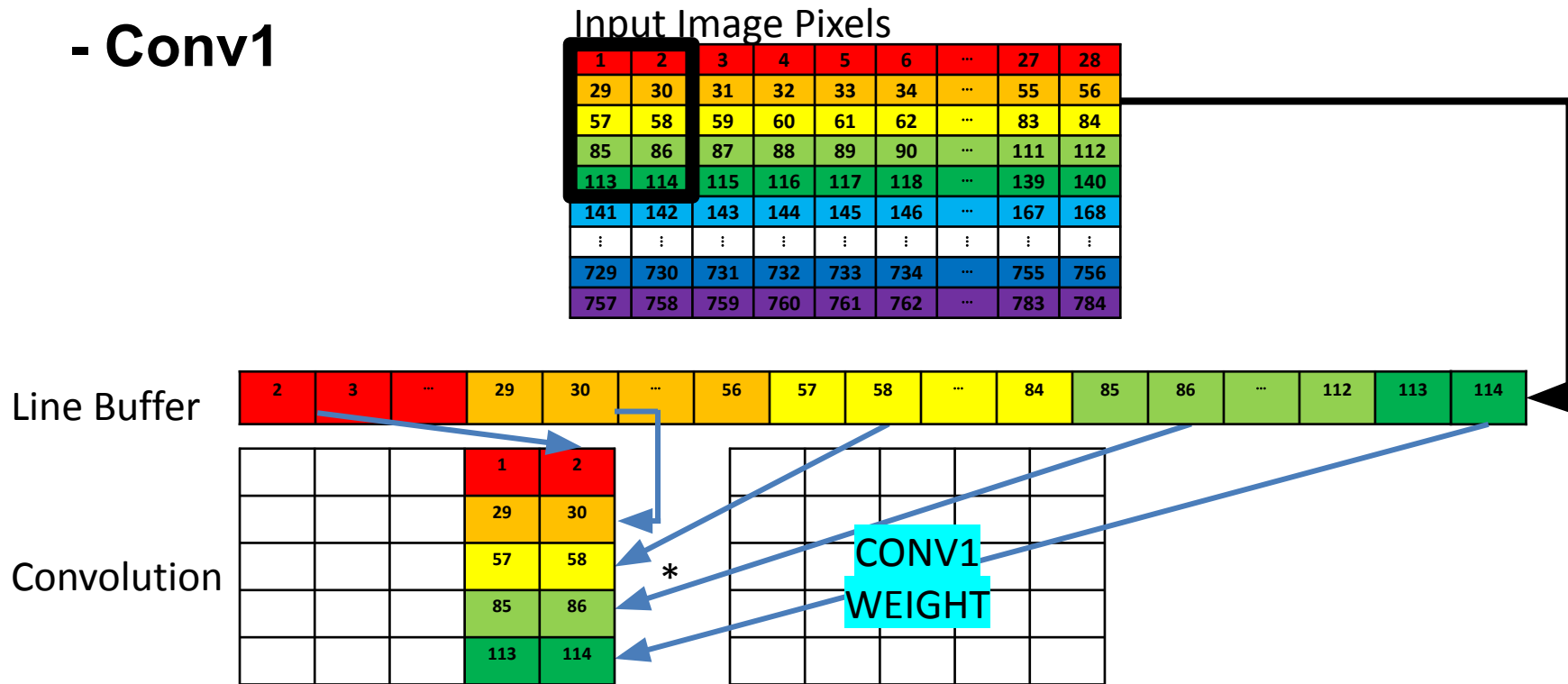
				1
				29
				57
				85
				113

*

CONV1
WEIGHT

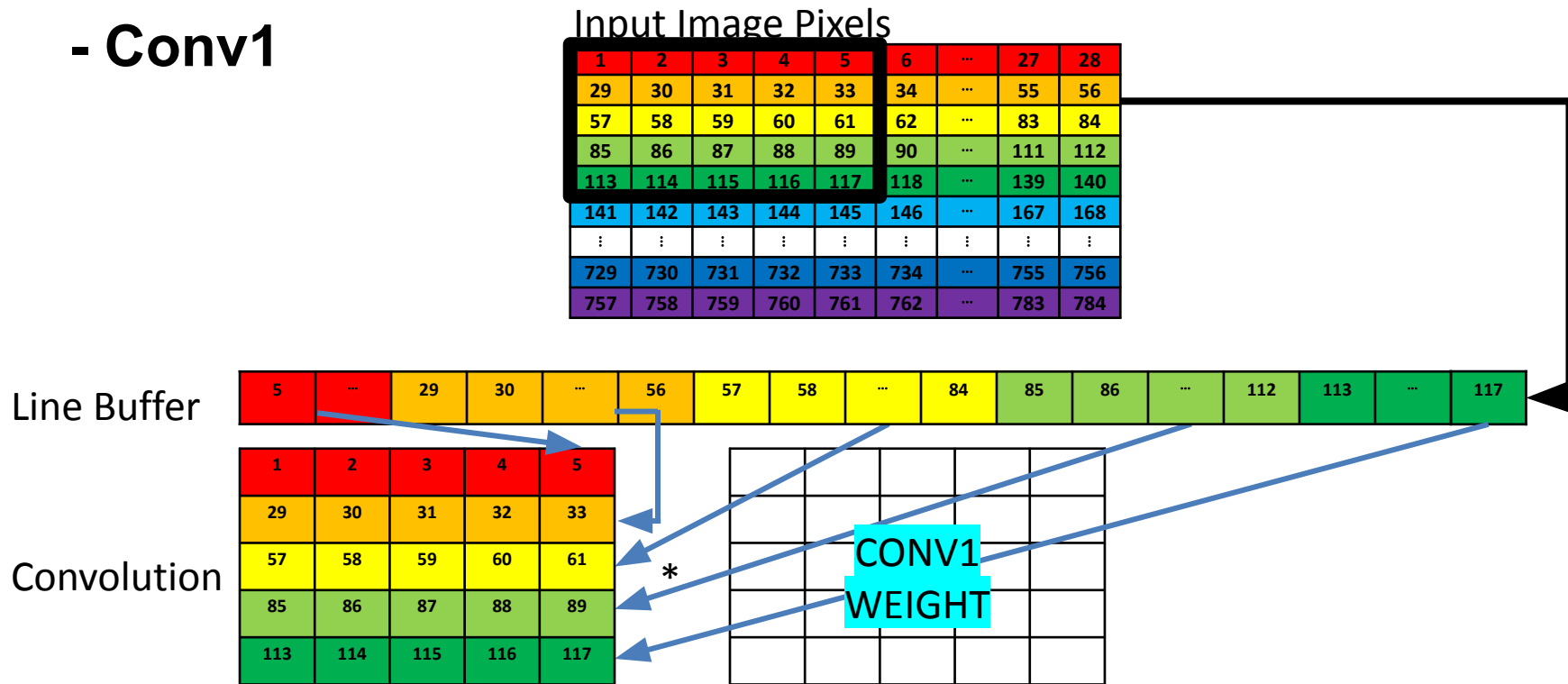
Method

Line buffer - Conv1



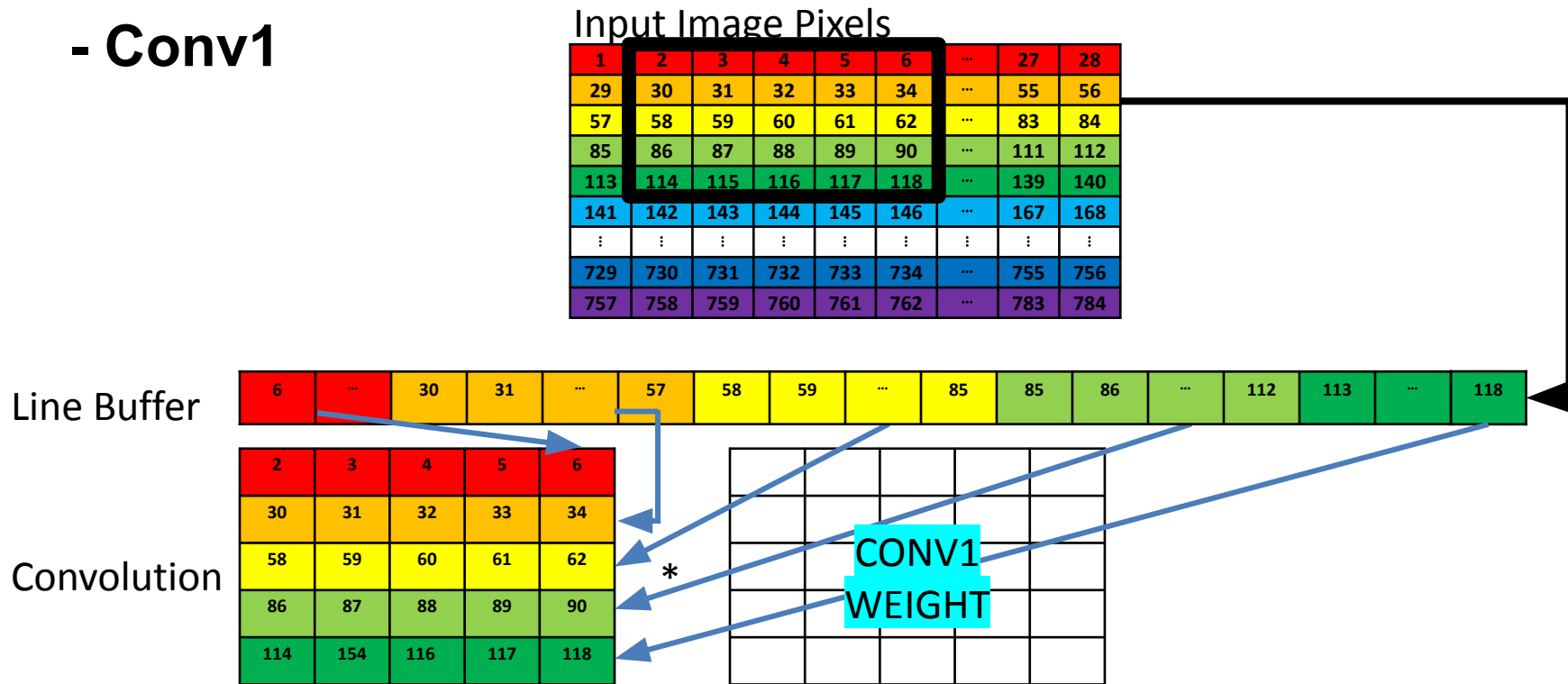
Method

Line buffer - Conv1



Method

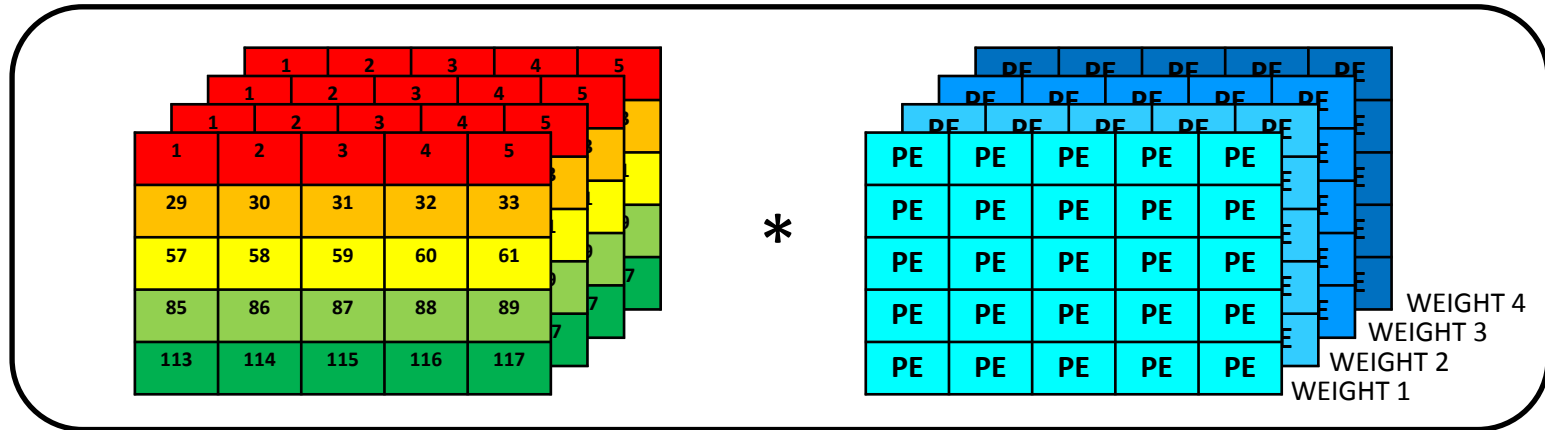
Line buffer - Conv1



Method

Line buffer - Conv1

PE ARRAY



In CONV1, a 5x5 input window generated by the line buffer is fed into the PE array, where four different filters are processed in parallel.

Method

Line buffer - Pooling

Conv1 Output

1	2	3	4	5	6	...	23	24
25	26	27	28	29	30	...	47	48
49	50	51	52	53	54	...	71	72
73	74	75	76	77	78	...	95	96
97	98	99	100	101	102	...	119	120
121	122	123	124	125	126	...	143	144
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
529	530	531	532	533	534	...	551	552
553	554	555	556	557	558	...	575	576

1. Only even rows fill the 24 line-buffer entries.

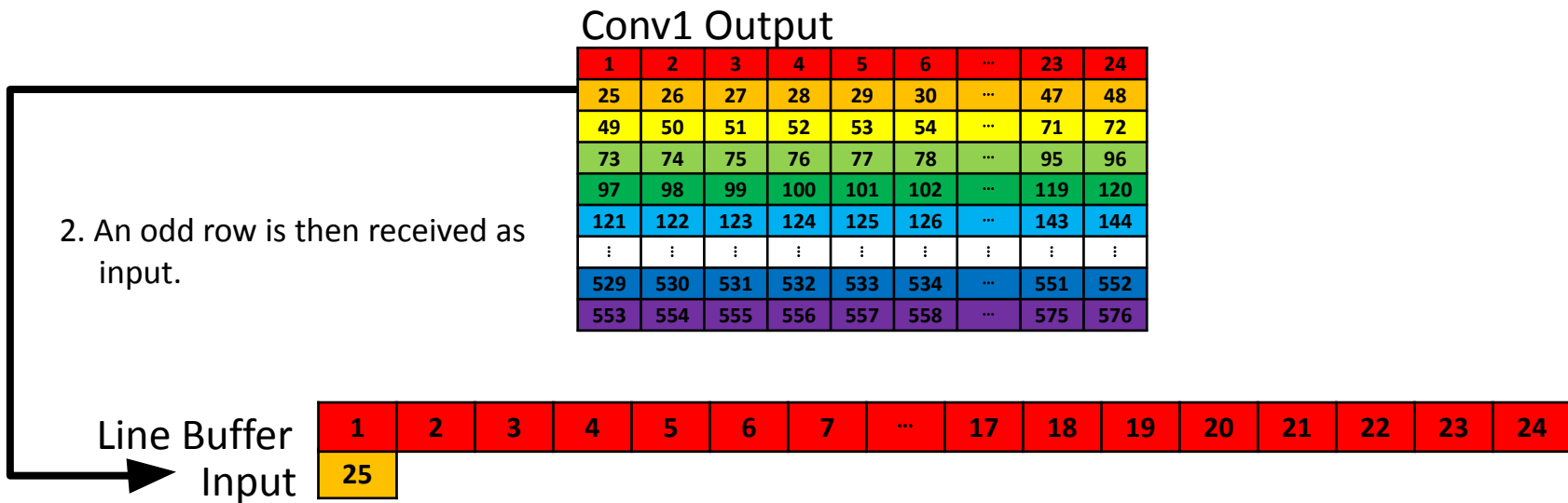
Line Buffer

1	2	3	4	5	6	7	...	17	18	19	20	21	22	23	24
---	---	---	---	---	---	---	-----	----	----	----	----	----	----	----	----

24

Method

Line buffer - Pooling



Method

Line buffer - Pooling

Conv1 Output

1	2	3	4	5	6	...	23	24
25	26	27	28	29	30	...	47	48
49	50	51	52	53	54	...	71	72
73	74	75	76	77	78	...	95	96
97	98	99	100	101	102	...	119	120
121	122	123	124	125	126	...	143	144
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
529	530	531	532	533	534	...	551	552
553	554	555	556	557	558	...	575	576

Line Buffer
Previous Input

1	2	3	4	5	6	7	...	17	18	19	20	21	22	23	24
25	26														

Current Input



Method

Line buffer - Pooling

Conv1 Output

1	2	3	4	5	6	...	23	24
25	26	27	28	29	30	...	47	48
49	50	51	52	53	54	...	71	72
73	74	75	76	77	78	...	95	96
97	98	99	100	101	102	...	119	120
121	122	123	124	125	126	...	143	144
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
529	530	531	532	533	534	...	551	552
553	554	555	556	557	558	...	575	576

Max pooling Input

Line Buffer
Previous Input

1	2
25	26

Current Input

3	4	5	6	7	...	17	18	19	20	21	22	23	24
---	---	---	---	---	-----	----	----	----	----	----	----	----	----

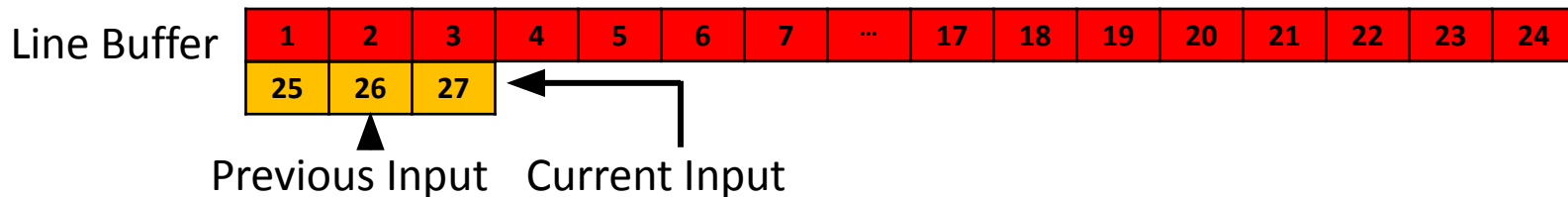
- For an odd row, the data are split into the previous input and the current input, and the pooling result is valid only when the current input corresponds to an odd column.

Method

Line buffer - Pooling

Conv1 Output

1	2	3	4	5	6	...	23	24
25	26	27	28	29	30	...	47	48
49	50	51	52	53	54	...	71	72
73	74	75	76	77	78	...	95	96
97	98	99	100	101	102	...	119	120
121	122	123	124	125	126	...	143	144
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
529	530	531	532	533	534	...	551	552
553	554	555	556	557	558	...	575	576



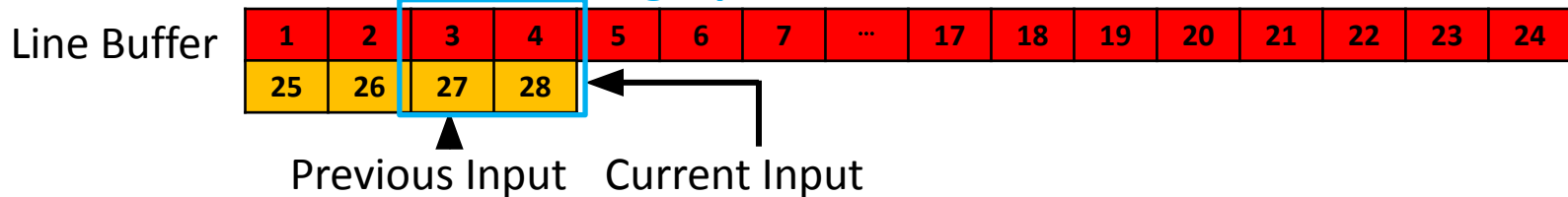
Method

Line buffer - Pooling

Conv1 Output

1	2	3	4	5	6	...	23	24
25	26	27	28	29	30	...	47	48
49	50	51	52	53	54	...	71	72
73	74	75	76	77	78	...	95	96
97	98	99	100	101	102	...	119	120
121	122	123	124	125	126	...	143	144
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
529	530	531	532	533	534	...	551	552
553	554	555	556	557	558	...	575	576

Max pooling Input



Method

Reconfigurable PE - Conv2

Conv2 Input

1	2	3	4	5	6	...	11	12
13	14	15	16	17	18	...	23	24
25	26	27	28	29	30	...	35	36
37	38	39	40	41	42	...	47	48
49	50	51	52	53	54	...	59	60
61	62	63	64	65	66	...	71	72
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
121	122	123	124	125	126	...	131	132
133	134	135	136	137	138	...	143	144

Line Buffer



$12 * 4$

CONV2 uses the same line-buffer structure as CONV1 in order to reuse the processing elements (PEs) used in CONV1.

Method

Reconfigurable PE - Conv2

Conv2 Input

1	2	3	4	5	6	...	11	12
13	14	15	16	17	18	...	23	24
25	26	27	28	29	30	...	35	36
37	38	39	40	41	42	...	47	48
49	50	51	52	53	54	...	59	60
61	62	63	64	65	66	...	71	72
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
121	122	123	124	125	126	...	131	132
133	134	135	136	137	138	...	143	144

Line Buffer

1	2	...	12	13	14	...	24	25	26	...					
---	---	-----	----	----	----	-----	----	----	----	-----	--	--	--	--	--

		CONV1		
		WEIGHT		

Since the weights are updated while the line buffer is being filled, the weight update latency can be hidden.

Method

Reconfigurable PE - Conv2

Conv2 Input

1	2	3	4	5	6	...	11	12
13	14	15	16	17	18	...	23	24
25	26	27	28	29	30	...	35	36
37	38	39	40	41	42	...	47	48
49	50	51	52	53	54	...	59	60
61	62	63	64	65	66	...	71	72
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
121	122	123	124	125	126	...	131	132
133	134	135	136	137	138	...	143	144

Line Buffer

1	2	...	12	13	14	...	24	25	26	...	36	37	38	...	48
---	---	-----	----	----	----	-----	----	----	----	-----	----	----	----	-----	----

		CONV2		
		WEIGHT		

Since the weights are updated while the line buffer is being filled, the weight update latency can be hidden.

Method

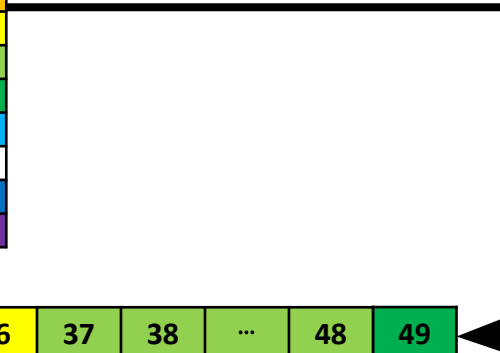
Reconfigurable PE - Conv2

Conv2 Input

1	2	3	4	5	6	...	11	12
13	14	15	16	17	18	...	23	24
25	26	27	28	29	30	...	35	36
37	38	39	40	41	42	...	47	48
49	50	51	52	53	54	...	59	60
61	62	63	64	65	66	...	71	72
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
121	122	123	124	125	126	...	131	132
133	134	135	136	137	138	...	143	144

Line Buffer

1	2	...	12	13	14	...	24	25	26	...	36	37	38	...	48	49
---	---	-----	----	----	----	-----	----	----	----	-----	----	----	----	-----	----	----



Method

Reconfigurable PE - Conv2

Conv2 Input

1	2	3	4	5	6	...	11	12
13	14	15	16	17	18	...	23	24
25	26	27	28	29	30	...	35	36
37	38	39	40	41	42	...	47	48
49	50	51	52	53	54	...	59	60
61	62	63	64	65	66	...	71	72
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
121	122	123	124	125	126	...	131	132
133	134	135	136	137	138	...	143	144

Line Buffer

1	2	...	12	13	14	...	24	25	26	...	36	37	38	...	48	49
---	---	-----	----	----	----	-----	----	----	----	-----	----	----	----	-----	----	----

Convolution

				1
				13
				25
				37
				49

Method

Reconfigurable PE - Conv2

Conv2 Input

1	2	3	4	5	6	...	11	12
13	14	15	16	17	18	...	23	24
25	26	27	28	29	30	...	35	36
37	38	39	40	41	42	...	47	48
49	50	51	52	53	54	...	59	60
61	62	63	64	65	66	...	71	72
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
121	122	123	124	125	126	...	131	132
133	134	135	136	137	138	...	143	144

Line Buffer

3	...	13	14	...	24	25	26	...	36	37	38	...	48	49	50	51
---	-----	----	----	-----	----	----	----	-----	----	----	----	-----	----	----	----	----

Convolution

		1	2	3
		13	14	15
		25	26	27
		37	38	39
		49	50	51

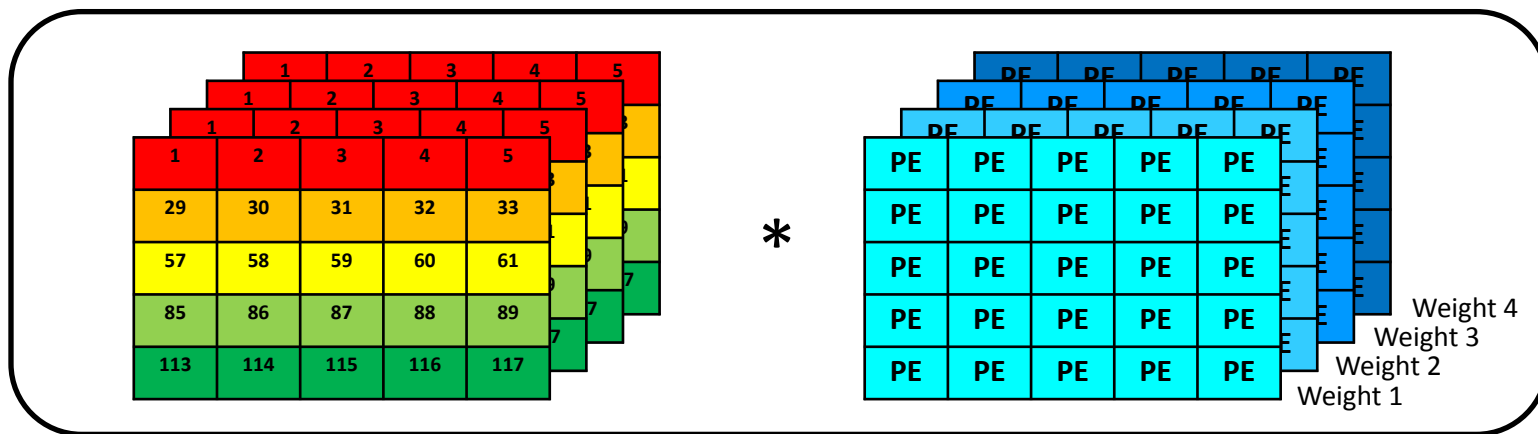
*

CONV2
WEIGHT

Method

Reconfigurable PE - Conv2

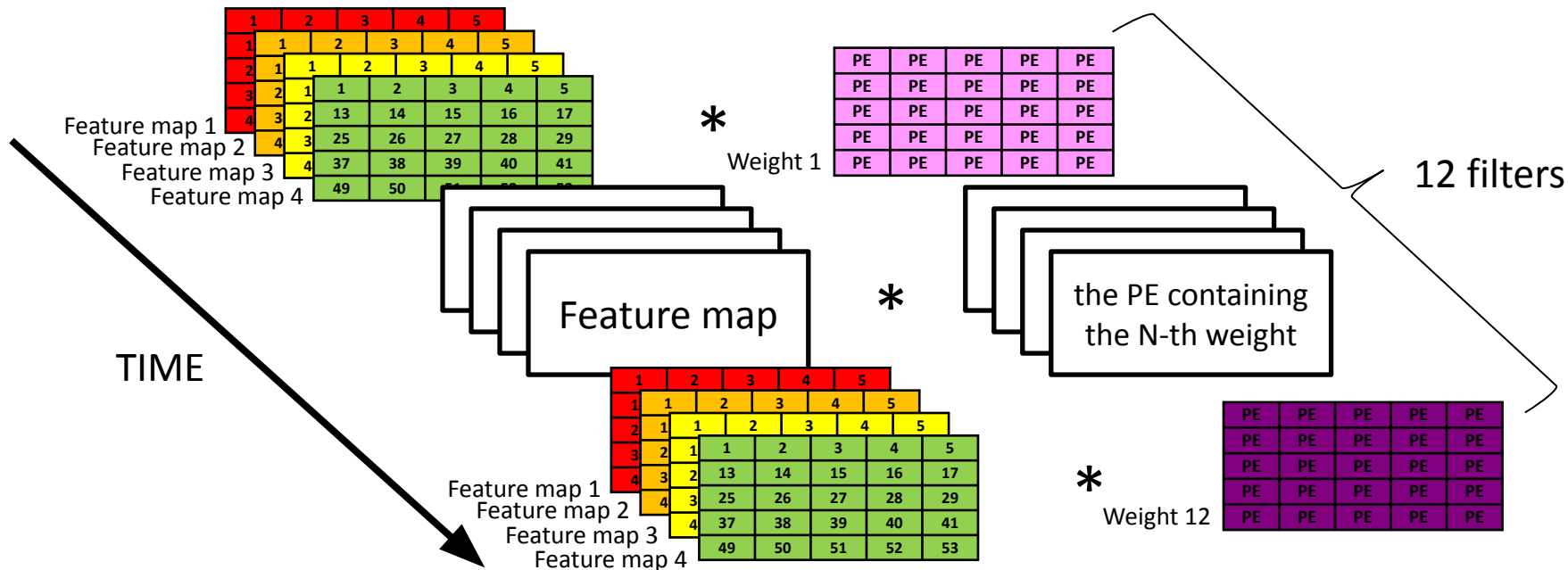
PE ARRAY



The same PE array is reused for both CONV1 and CONV2.

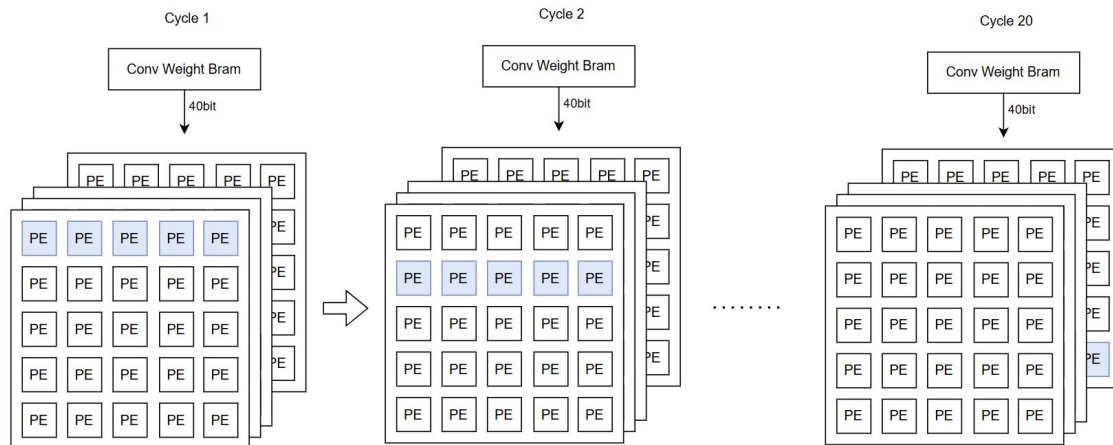
Method

Reconfigurable PE - Conv2

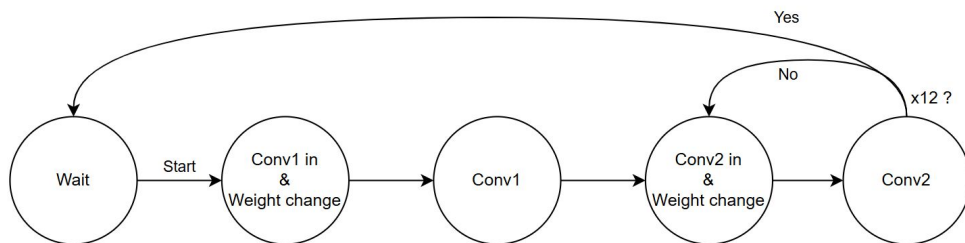


Method

PE Flow

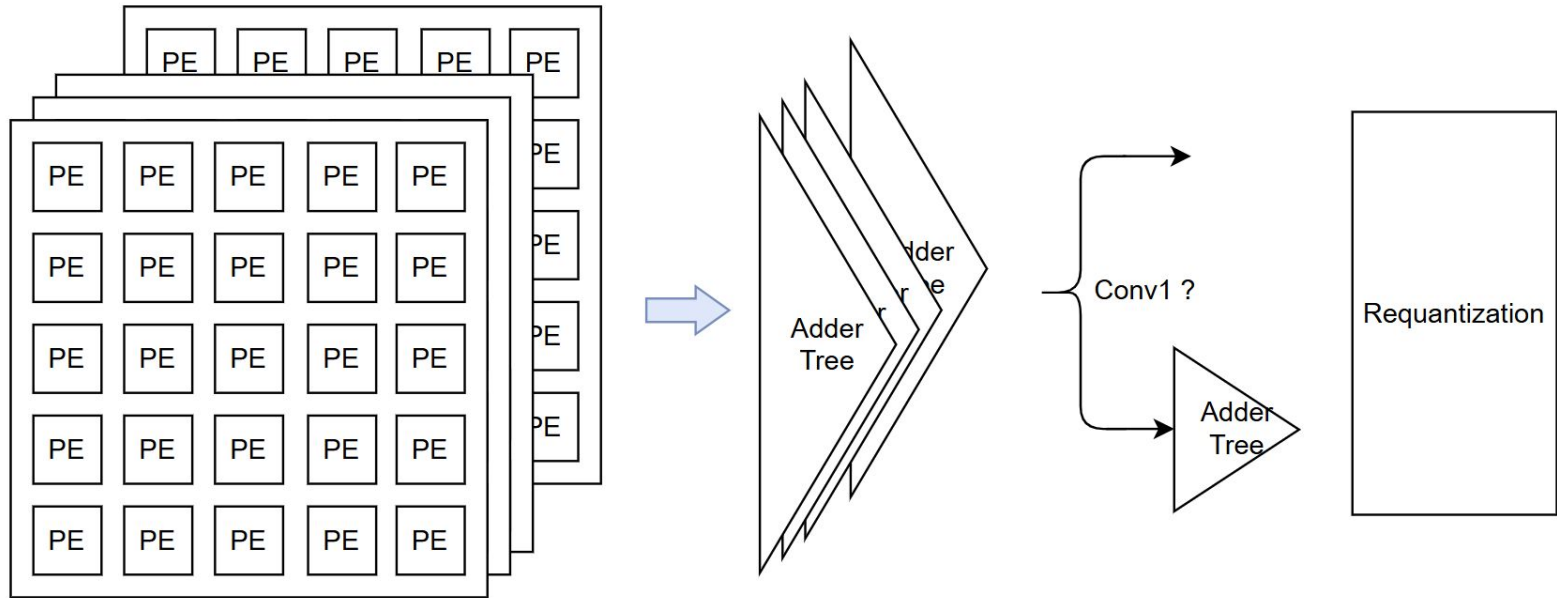


Conv1	Conv2	Weight
$28*4+5=117$	$12*4+5=53$	20



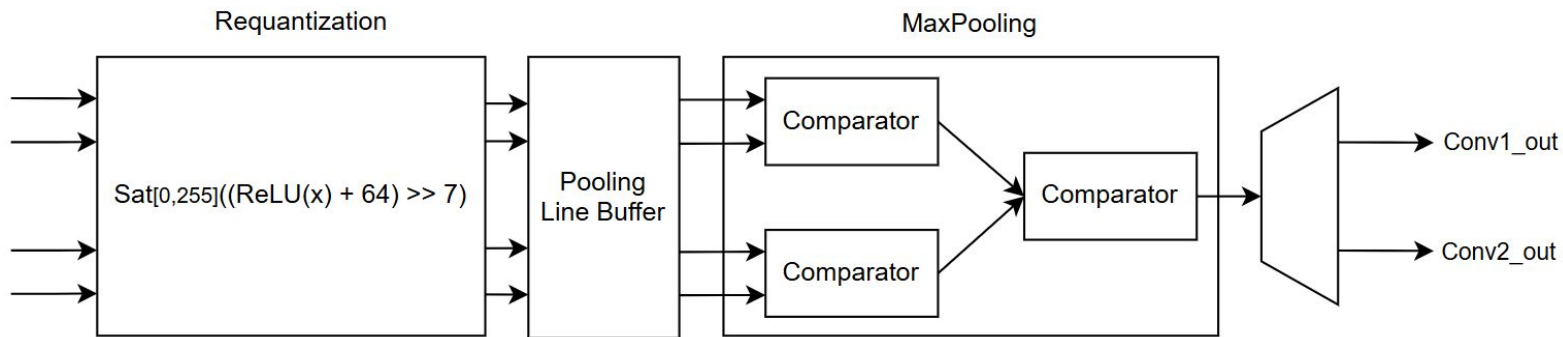
Method

PE Flow



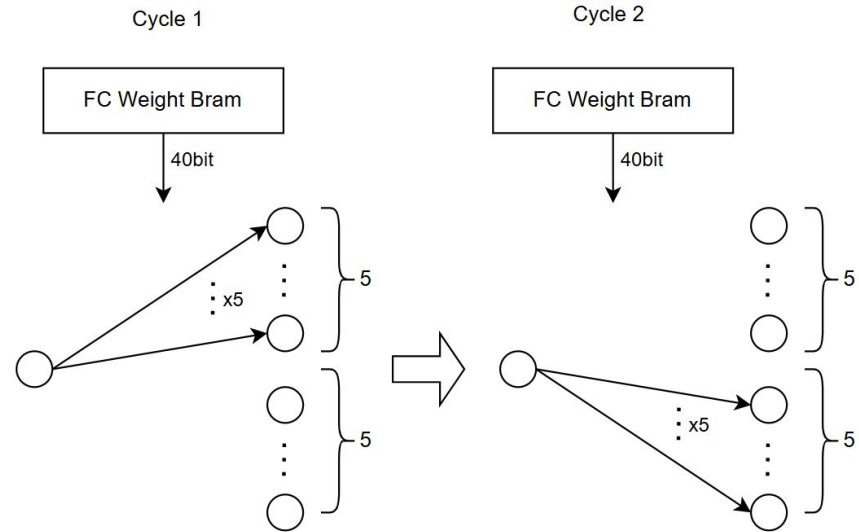
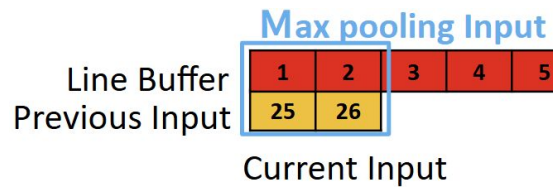
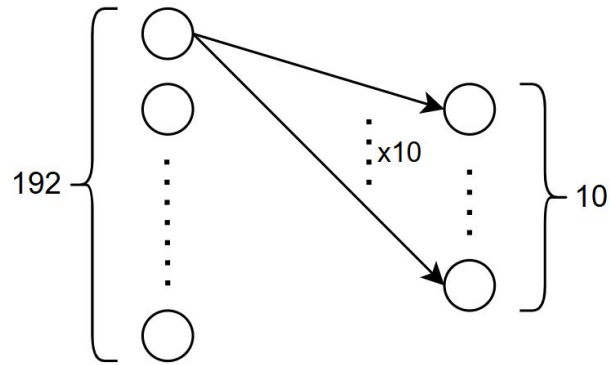
Method

Requantization & Max Pooling



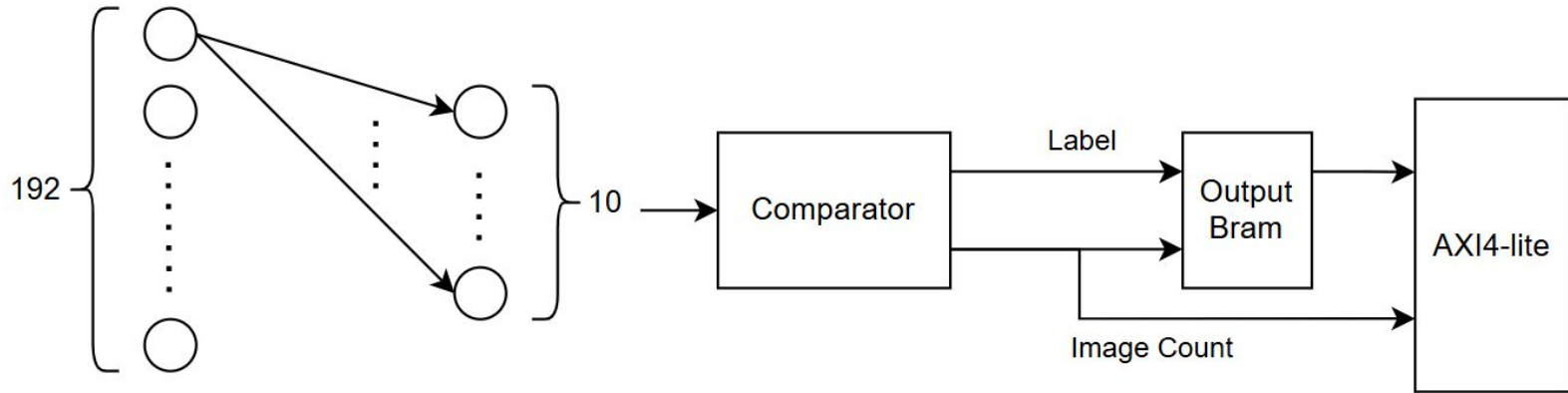
Method

FC layer & Comparator



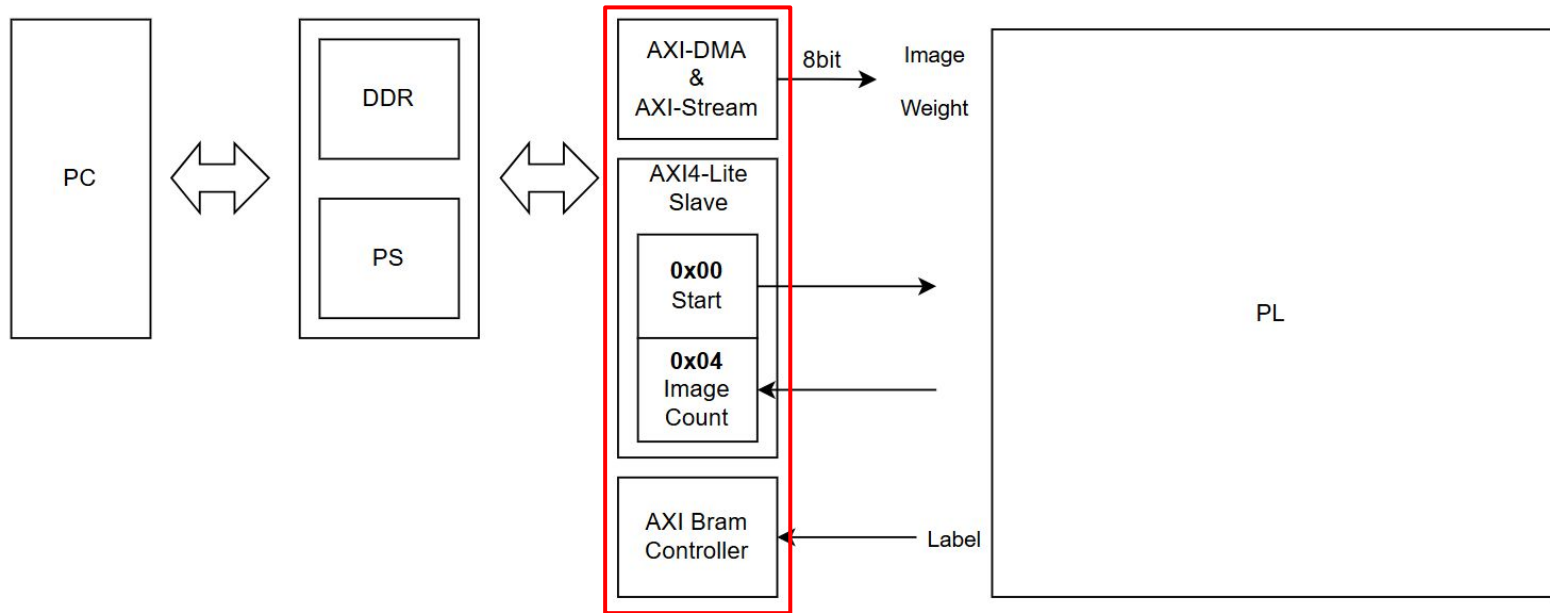
Method

FC layer & Comparator



Method

AXI Interface

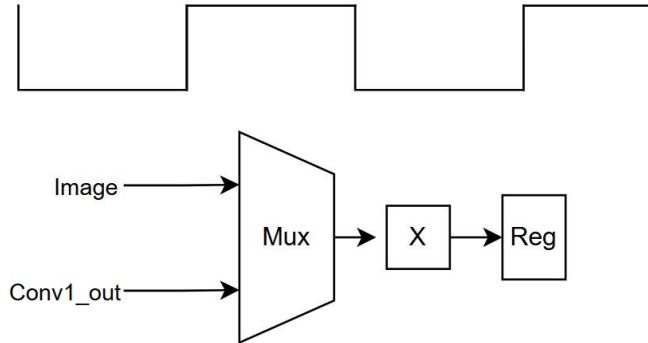


Method

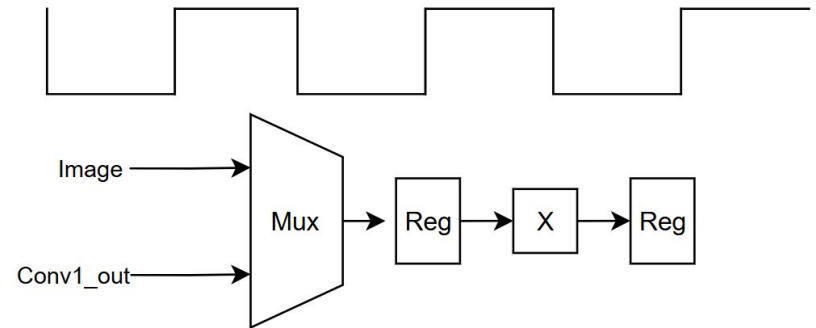
Pipelining

Name	Slack	^1	Levels	High Fanout	From	To
Path 1	-1.822		3	4	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[0].ROW[4].COL[0].u_pe/p_out_reg[7]/D
Path 2	-1.821		3	4	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[0].ROW[4].COL[0].u_pe/p_out_reg[11]/D
Path 3	-1.813		3	4	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[0].ROW[4].COL[0].u_pe/p_out_reg[15]/D
Path 4	-1.750		3	4	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[0].ROW[4].COL[0].u_pe/p_out_reg[0]/D
Path 5	-1.746		3	4	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[0].ROW[4].COL[0].u_pe/p_out_reg[6]/D
Path 6	-1.745		3	4	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[0].ROW[4].COL[0].u_pe/p_out_reg[10]/D
Path 7	-1.743		3	4	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[0].ROW[4].COL[0].u_pe/p_out_reg[9]/D
Path 8	-1.713		3	2	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[2].ROW[4].COL[0].u_pe/p_out_reg[3]/D
Path 9	-1.712		3	4	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[0].ROW[4].COL[0].u_pe/p_out_reg[13]/D
Path 10	-1.710		3	2	<hidden>	design_1_u1TOP_1/inst/PE_array_top_0/ARRAY[2].ROW[4].COL[0].u_pe/p_out_reg[4]/D

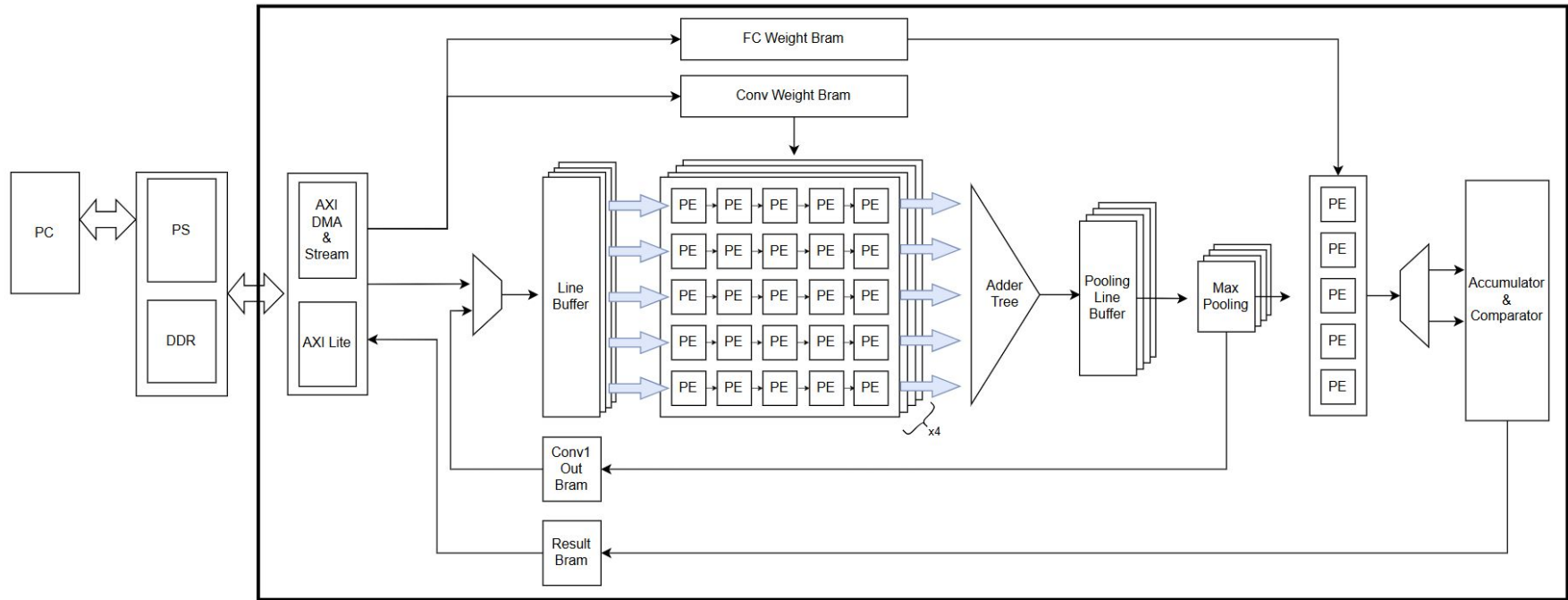
100Mhz ~~



200Mhz



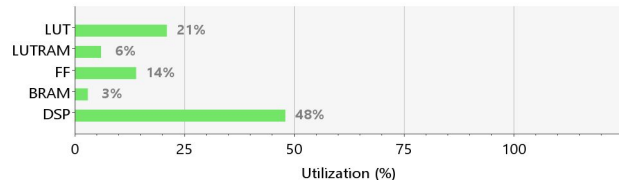
System Architecture



FPGA Implementation Results

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	D S P 1	Bonded IOPADs (130)	BUFGCTRL (32)
▼ N design_1_wrapper	10925	15310	194	4939	9808	1117	4	105	130	1
▼ I design_1_i (design_1)	10925	15310	194	4939	9808	1117	4	105	0	1
▼ I myip_v1_0_0 (design_1)	4479	8415	193	2574	4280	199	0	105	0	0
▼ I inst (design_1_myip_v1_0_0)	4479	8310	193	2556	4280	199	0	105	0	0
> I TOP_0 (design_1_wrapper)	4434	8222	193	2532	4235	199	0	105	0	0

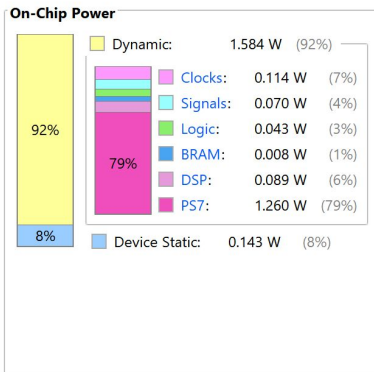
Resource	Utilization	Available	Utilization %
LUT	10925	53200	20.54
LUTRAM	1117	17400	6.42
FF	15310	106400	14.39
BRAM	4	140	2.86
DSP	105	220	47.73



FPGA Implementation Results

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 1.727 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 44.9°C
Thermal Margin: 40.1°C (3.3 W)
Ambient Temperature: 25.0 °C
Effective θ_{JA} : 11.5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Medium
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.029 ns	Worst Hold Slack (WHS): 0.020 ns	Worst Pulse Width Slack (WPWS): 1.250 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 46782	Total Number of Endpoints: 46782	Total Number of Endpoints: 17136

All user specified timing constraints are met.

200Mhz



FPGA Implementation Results

=== Summary ===

PS Acc = 996/1000 = 99.60% | Total = 8665609.56 us | Avg/img = 8665.61 us

PL Acc = 996/1000 = 99.60% | Weight upload = 39.58 us

PL Inference only: Total = 15482.21 us | Avg/img = 15.48 us

PL Cold-start total (upload + infer) = 15521.79 us

Speedup (PS/PL, steady-state avg) = 559.71x

